

Remise à niveau Bash

Cahier d'exercices — environ 2 heures

Navigation & fichiers · Redirections · Variables · Expansions ·
Structures de contrôle · Tableaux · Arithmétique · Processus

Consignes générales : chaque exercice se résout dans un terminal Bash. Créez un dossier de travail dédié (`mkdir ~/exo-bash && cd ~/exo-bash`) avant de commencer. Les corrigés se trouvent en fin de document — essayez d'abord sans les consulter.

Sommaire

- 1. Prise en main & navigation (10 min)
- 2. Fichiers, permissions & propriétaires (15 min)
- 3. Recherche de texte : grep, find, cut, sort, uniq (15 min)
- 4. Redirections et pipelines (12 min)
- 5. Variables, environnement & variables spéciales (15 min)
- 6. Expansions : chaînes, tilde, globbing (15 min)
- 7. Tests et expressions conditionnelles (10 min)
- 8. Structures de contrôle (20 min)
- 9. Arithmétique, tableaux & dictionnaires (18 min)
- 10. Processus, signaux & archives (12 min)
- 11. Projets de synthèse (18 min)
- Corrigés

1. Prise en main & navigation

Commandes ciblées : `ls`, `cd`, `pwd`, `cat`, `echo`, `wc`, `head`, `tail`, `file`, `stat`.

Exercice 1 — Exploration d'un répertoire

■ 5 min · Difficulté : facile

Dans votre dossier de travail, créez trois fichiers `a.txt`, `b.txt`, `.cache` (fichier caché) avec la commande `touch`. Affichez :

- la liste simple du dossier
- la liste incluant les fichiers cachés
- la liste détaillée (permissions, taille, date) avec les inodes

Combien de fichiers voyez-vous dans chaque cas ? Notez la différence entre `ls -l` et `ls -li`.

Exercice 2 — Chemins et métadonnées

■ 5 min · Difficulté : facile

Depuis votre `$HOME`, déplacez-vous dans votre dossier d'exercices, revenez au répertoire précédent avec `cd -`, puis affichez le chemin absolu courant.

Créez un fichier `notes.txt` contenant trois lignes de texte au choix (utilisez `cat > notes.txt` puis `Ctrl+D`). Affichez :

- le nombre de lignes du fichier
- le type du fichier (via `file -b`)
- sa taille en octets via `stat` avec un format personnalisé

2. Fichiers, permissions & propriétaires

Commandes ciblées : `cp`, `mv`, `rm`, `mkdir`, `ln`, `chmod`, `chown`, `sudo`, `su`.

Exercice 3 — Copier, lier, organiser

■ 6 min · Difficulté : facile

Créez une arborescence `projet/src` et `projet/backup` en une seule commande `mkdir`. Copiez tous les fichiers `.txt` du dossier courant vers `projet/src`, puis créez un lien symbolique `projet/latest.txt` pointant vers l'un des fichiers copiés.

Renommez ensuite `projet` en `projet_v1` sans écraser de fichier existant par erreur (option de confirmation).

Exercice 4 — Droits d'accès

■ 7 min · Difficulté : moyen

Créez un script vide `deploy.sh`. Rendez-le exécutable uniquement pour son propriétaire, lisible et exécutable pour le groupe, et sans aucun droit pour les autres — d'abord en notation symbolique, puis vérifiez le résultat, puis reproduisez le même réglage en notation octale.

Quelle est la valeur octale obtenue ? Expliquez pourquoi.

Exercice 5 — Propriétaires et élévation de privilèges

■ 7 min · Difficulté : moyen

Sans exécuter réellement les commandes (elles nécessitent `root`), écrivez les commandes qui permettraient de :

- donner le fichier `index.html` à l'utilisateur `www-data` sans changer le groupe
- changer récursivement propriétaire et groupe de `/var/www` en `ubuntu:www-data`
- exécuter `apt update` avec les droits root
- ouvrir un shell root persistant, puis un shell root avec l'environnement complet de root (`su -`)

3. Recherche de texte : grep, find, cut, sort, uniq

Un bon réflexe Bash : combiner ces outils pour explorer rapidement des fichiers texte.

Exercice 6 — Journal de connexions (grep)

■ 8 min · Difficulté : moyen

Créez un fichier `acces.log` avec ce contenu :

```
2024-01-10 10:22 alice LOGIN OK
2024-01-10 10:23 bob LOGIN FAIL
2024-01-10 10:25 alice LOGOUT
2024-01-11 08:01 carol LOGIN OK
2024-01-11 08:05 bob LOGIN FAIL
2024-01-11 09:10 bob LOGIN OK
```

Écrivez les commandes `grep` permettant de :

- afficher toutes les lignes contenant un échec de connexion, avec leur numéro de ligne
- afficher toutes les lignes concernant bob, insensible à la casse
- compter le nombre de connexions réussies (`LOGIN OK`) avec une expression étendue qui accepte aussi `login ok` en minuscules

Exercice 7 — Recherche de fichiers (find)

■ 6 min · Difficulté : moyen

Dans votre arborescence d'exercices, recherchez :

- tous les fichiers réguliers se terminant par `.sh`
- tous les fichiers dont le nom commence par `backup`, quel que soit le sous-dossier
- tous les éléments qui sont des dossiers uniquement

Exercice 8 — Extraction et tri d'un CSV

■ 8 min · Difficulté : moyen

Créez un fichier `data.csv` :

```
nom,age,ville
Alice,34,Paris
Bob,28,Lyon
Carol,45,Paris
David,19,Marseille
```

Réalisez, avec `cut`, `sort` et `uniq` :

- l'extraction de la seule colonne des villes (sans l'en-tête)
- la liste des villes uniques triées par ordre alphabétique
- le tri du fichier complet par âge décroissant (indice : trier les données seules, sans l'en-tête, avec l'option `numeric`)

4. Redirections et pipelines

Commandes ciblées : `|`, `&`, `>`, `>>`, `<`, `2>`, `2>&1`, `/dev/null`, `exec`, `tee`.

Exercice 9 — Rediriger sortie et erreurs

■ 7 min · Difficulté : moyen

Exécutez une commande qui échoue volontairement, par exemple `ls dossier_inexistant`. Écrivez une version de cette commande qui :

- envoie uniquement le message d'erreur dans un fichier `erreurs.log` (en ajout, sans écraser)
- envoie la sortie standard ET la sortie d'erreur dans le même fichier `tout.log`
- supprime totalement le message d'erreur affiché à l'écran

Exercice 10 — Pipeline et tee

■ 8 min · Difficulté : moyen

À partir du fichier `data.csv` de l'exercice 8, construisez un pipeline en une seule ligne qui : extrait la colonne des noms, la trie, puis affiche le résultat à l'écran ET l'enregistre dans `noms_tries.txt` en même temps (sans utiliser deux commandes séparées).

Bonus : utilisez le descripteur de fichier 3 pour rediriger la sortie d'une commande vers `audit.log` via `exec`, puis écrivez dedans avec `echo texte >&3`.

5. Variables, environnement & variables spéciales

Commandes ciblées : `export`, `unset`, `readonly`, `declare`, `set`, `env`, `alias`, `$?`, `$$`, `$!`, `$@`, `$0`, `$#`.

Exercice 11 — Script et arguments

■ 8 min · Difficulté : moyen

Écrivez un script `infos.sh` qui, lorsqu'on l'appelle avec des arguments (ex : `./infos.sh a b c`), affiche :

- le nom du script
- le nombre d'arguments reçus
- le premier et le dernier argument (sans en connaître le nombre exact à l'avance)
- tous les arguments sous forme de liste, un par ligne

Indice : combinez `$0`, `$#`, `$@` et une boucle.

Exercice 12 — Codes de retour et processus

■ 7 min · Difficulté : moyen

Lancez une commande qui réussit puis affichez `$?`. Faites de même avec une commande qui échoue. Ensuite :

- lancez `sleep 5 &` en arrière-plan et affichez immédiatement son PID via la variable spéciale dédiée
- affichez le PID du shell courant
- utilisez `wait` pour attendre la fin du processus en arrière-plan avant d'afficher un message de fin

Exercice 13 — Variables d'environnement et export

■ 8 min · Difficulté : moyen

Créez une variable locale `SECRET=1234`. Écrivez un petit script qui affiche cette variable via `echo $SECRET` et exécutez-le : que se passe-t-il et pourquoi ?

Corrigez le problème pour que le script voie la variable, sans la définir directement dans le script. Ensuite, rendez une variable `VERSION="1.0"` constante avec `readonly` et vérifiez qu'une tentative de modification échoue.

Enfin, créez un alias `ll` pour `ls -la`, testez-le, puis supprimez-le.

6. Expansions : chaînes, tilde, globbing

Commandes ciblées : `${#str}`, `${var:-word}`, `${var#pattern}`, `${var/pattern/repl}`, `~`, `*`, `?`, `{a..b}`.

Exercice 14 — Manipulation de chaînes

■ 10 min · Difficulté : moyen

Avec `chemin="/home/alice/projet/rapport.final.txt"`, utilisez uniquement les expansions de paramètres (pas de commande externe) pour obtenir :

- la longueur de la chaîne
- le nom de fichier seul (`rapport.final.txt`), en retirant le plus long préfixe se terminant par `/`
- l'extension seule (`txt`), en retirant le plus court préfixe jusqu'au dernier point... puis réfléchissez : que donne le retrait du plus *long* préfixe jusqu'à un point ? Comparez les deux
- le chemin sans son extension, en retirant le plus court suffixe commençant par un point
- la chaîne avec `alice` remplacé par `bob`
- la chaîne entièrement en majuscules

Exercice 15 — Valeurs par défaut

■ 7 min · Difficulté : moyen

Dans un script, une variable `ENV` peut être définie ou non par l'utilisateur. Écrivez une ligne qui affiche la valeur de `ENV` si elle est définie, ou `"production"` sinon, sans modifier `ENV`.

Écrivez ensuite une ligne qui affecte `"production"` à `ENV` si elle n'est pas définie (et la garde sinon). Enfin, écrivez une ligne qui arrête le script avec un message d'erreur explicite si une variable `API_KEY` n'est pas définie.

Exercice 16 — Motifs de fichiers (globbing)

■ 8 min · Difficulté : moyen

Dans un dossier contenant les fichiers `log1.txt` `log2.txt` `log10.txt` `rapport.csv` `image.png` `notes.md`, écrivez des motifs shell (sans `find`) pour lister :

- tous les fichiers `.txt`
- uniquement `log1.txt` et `log2.txt` (pas `log10`)
- tous les fichiers dont l'extension est `csv` ou `md`
- les fichiers dont le nom ne commence pas par la lettre `l`

Créez ensuite d'un coup les dossiers `archive_2022`, `archive_2023` et `archive_2024` grâce à l'expansion d'accolades.

7. Tests et expressions conditionnelles

Commandes ciblées : `test`, `[ ]`, `[ [ ] ]`, `-z`, `-f`, `-d`, `-e`, `-eq`, `=~`.

Exercice 17 — Tests sur fichiers et chaînes

■ 6 min · Difficulté : facile

Écrivez des tests (avec `[[ ]]`) répondant à :

- le fichier `config.yml` existe-t-il ET est-il inscriptible ?
- le dossier `logs` existe-t-il ?
- la variable `NOM` est-elle vide ?
- deux variables entières `A` et `B` sont-elles égales ?

Exercice 18 — Validation par expression régulière

■ 9 min · Difficulté : difficile

Écrivez un test `[[ ... =~ ... ]]` qui vérifie qu'une variable `email` ressemble à une adresse mail simple de la forme `texte@texte.texte` (lettres, chiffres, points, tirets).

Écrivez un second test qui vérifie qu'une variable `code_postal` contient exactement 5 chiffres.

8. Structures de contrôle

Commandes ciblées : `if/elif/else`, `for`, `while`, `until`, `case`, `select`, fonctions.

Exercice 19 — Classification par condition

■ 8 min · Difficulté : moyen

Écrivez un script `note.sh` prenant une note sur 20 en argument et affichant "Excellent" (≥ 16), "Bien" (≥ 12), "Passable" (≥ 10) ou "Insuffisant" (sinon), via un `if/elif/else`.

Exercice 20 — Boucles for

■ 8 min · Difficulté : moyen

Écrivez une boucle `for ... in` qui affiche la table de multiplication de 7 (de 7×1 à 7×10).

Écrivez une boucle `for ( ( ; ; ) )` de style C qui affiche uniquement les nombres pairs entre 0 et 20.

Écrivez une boucle `for` qui parcourt tous les fichiers `.txt` du dossier courant et affiche leur nombre de lignes.

Exercice 21 — while, until et menu select

■ 10 min · Difficulté : difficile

Écrivez un compteur avec `while` qui affiche les nombres de 1 à 5, puis le même résultat avec `until`.

Écrivez un menu interactif avec `select` proposant les choix Démarrer, Arrêter, Quitter. Le menu doit afficher le choix sélectionné et se terminer proprement quand l'utilisateur choisit "Quitter" (indice : `break` et la variable `$REPLY`).

Exercice 22 — case et fonctions

■ 10 min · Difficulté : difficile

Écrivez une fonction `saluer` qui prend un prénom en paramètre et affiche "Bonjour <prénom> !".

Écrivez ensuite un bloc `case` qui, à partir d'une variable `extension` (par exemple `txt`, `jpg`, `png`, `pdf`), affiche la catégorie du fichier ("document", "image" ou "inconnu"), en regroupant `jpg` et `png` dans le même motif.

9. Arithmétique, tableaux & dictionnaires

Commandes ciblées : `$( ( ))`, tableaux indexés, tableaux associatifs (`declare -A`).

Exercice 23 — Calculs arithmétiques

■ 6 min · Difficulté : facile

En utilisant uniquement `$( ( ) )`, calculez :

- le reste de la division de 17 par 5
- 2 puissance 10
- la moyenne entière de trois notes 12, 15 et 9
- incrémentez une variable `compteur` initialisée à 0, trois fois de suite

Exercice 24 — Tableaux indexés

■ 9 min · Difficulté : moyen

Créez un tableau `fruits=("pomme" "banane" "cerise")`. Écrivez les commandes pour :

- afficher le deuxième élément
- afficher tous les éléments
- afficher le nombre d'éléments
- ajouter "kiwi" à la fin
- supprimer l'élément "banane"
- parcourir le tableau avec une boucle `for` et afficher chaque élément numéroté

Exercice 25 — Tableau associatif

■ 9 min · Difficulté : difficile

Déclarez un tableau associatif `capitaLes` avec au moins trois paires pays/capitale (France/Paris, Italie/Rome, Espagne/Madrid). Écrivez les commandes pour :

- afficher la capitale de l'Italie
- afficher la liste de tous les pays (les clés)
- afficher le nombre d'entrées
- parcourir le tableau et afficher chaque ligne sous la forme "France -> Paris"
- supprimer l'entrée "Espagne"

10. Processus, signaux & archives

Commandes ciblées : `ps`, `kill`, `trap`, `tar`, `gzip`, `zip`.

Exercice 26 — Signaux et nettoyage (`trap`)

■ 9 min · Difficulté : difficile

Écrivez un script qui lance `sleep 30` et qui, si on l'interrompt avec `Ctrl+C` (signal `SIGINT`), affiche "Interruption détectée, nettoyage..." avant de quitter, au lieu de s'arrêter brutalement. Testez-le, puis désactivez le comportement personnalisé pour revenir au comportement par défaut.

Exercice 27 — Sauvegarde compressée

■ 7 min · Difficulté : moyen

Créez trois fichiers dans un dossier `site`. Réalisez :

- une archive `site.tar.gz` du dossier
- l'affichage du contenu de cette archive sans l'extraire

- l'extraction de l'archive dans un nouveau dossier `site_restore`

11. Projets de synthèse

Deux mini-projets combinant plusieurs notions du cahier.

Exercice 28 — Rapport d'utilisateurs (synthèse)

■ 10 min · Difficulté : difficile

Créez un fichier `users.csv` :

```
nom,role,actif
alice,admin,oui
bob,user,non
carol,user,oui
david,admin,oui
```

Écrivez un script `rapport_users.sh` qui :

- lit le fichier ligne par ligne (sans l'en-tête) et stocke chaque utilisateur actif dans un tableau
- compte, avec un tableau associatif, le nombre d'utilisateurs par rôle parmi les actifs uniquement
- affiche à la fin un résumé du type : "admin: 2 actif(s)", "user: 1 actif(s)"
- termine avec un code de retour 1 si aucun utilisateur actif n'a été trouvé, 0 sinon

Exercice 29 — Surveillance de dossier (synthèse)

■ 10 min · Difficulté : difficile

Écrivez un script `surveille.sh` qui prend un dossier en argument et qui :

- vérifie que l'argument a bien été fourni et que le dossier existe, sinon affiche un message d'erreur clair sur la sortie d'erreur standard et quitte avec un code non nul
- compte le nombre total de fichiers, le nombre de fichiers `.log`, et la taille totale approximative du dossier
- si au moins un fichier `.log` contient le mot "ERROR" (recherche insensible à la casse), archive tout le dossier dans une archive `tar.gz` nommée avec la date du jour, et affiche un message le confirmant
- sinon affiche simplement "Aucune erreur détectée"

Corrigés

Solutions proposées — d'autres réponses correctes existent souvent.

Exercice 1 — Exploration d'un répertoire

```
touch a.txt b.txt .cache
ls
ls -a
ls -li
# ls -a montre 5 entrées (a.txt, b.txt, .cache, . et ..)
# ls -li ajoute le numéro d'inode en première colonne
```

Exercice 2 — Chemins et métadonnées

```
cd ~/exo-bash
cd -
pwd
cat > notes.txt
ligne1
ligne2
ligne3
<Ctrl+D>
wc -l notes.txt
file -b notes.txt
stat -c "%s octets" notes.txt
```

Exercice 3 — Copier, lier, organiser

```
mkdir -p projet/src projet/backup
cp *.txt projet/src/
ln -s projet/src/a.txt projet/latest.txt
mv -i projet projet_v1
```

Exercice 4 — Droits d'accès

```
touch deploy.sh
chmod u=rwx,g=rx,o= deploy.sh
ls -l deploy.sh
chmod 750 deploy.sh
# 750 = rwx (7) pour u, r-x (5) pour g, --- (0) pour o
```

Exercice 5 — Propriétaires et élévation de privilèges

```
chown www-data index.html
chown -R ubuntu:www-data /var/www
sudo apt update
sudo -i
su -
```

Exercice 6 — Journal de connexions (grep)

```
grep -n "FAIL" acces.log
grep -i "bob" acces.log
grep -Ec "([Ll][Oo][Gg][Ii][Nn] [Oo][Kk])" acces.log
```

Exercice 7 — Recherche de fichiers (find)

```
find . -type f -name "*.sh"
find . -name "backup*"
find . -type d
```

Exercice 8 — Extraction et tri d'un CSV

```
cut -sd "," -f 3 data.csv | tail -n +2
cut -sd "," -f 3 data.csv | tail -n +2 | sort -u
tail -n +2 data.csv | sort -t "," -k2 -nr
```

Exercice 9 — Rediriger sortie et erreurs

```
ls dossier_inexistant 2>> erreurs.log
ls dossier_inexistant &> tout.log
ls dossier_inexistant 2>/dev/null
```

Exercice 10 — Pipeline et tee

```
cut -sd "," -f 1 data.csv | tail -n +2 | sort | tee noms_tries.txt

exec 3> audit.log
echo "ligne d'audit" >&3
```

Exercice 11 — Script et arguments

```
#!/bin/bash
echo "Script: $0"
echo "Nombre d'arguments: $#"
```

```
echo "Premier: $1"
echo "Dernier: ${@: -1}"
for arg in "$@"; do
    echo "$arg"
done
```

Exercice 12 — Codes de retour et processus

```
true; echo $?
false; echo $?
sleep 5 &
echo "PID en arrière-plan: $!"
echo "PID du shell: $$"
wait $!
echo "Fin"■
```

Exercice 13 — Variables d'environnement et export

```

SECRET=1234
cat > test.sh <<'EOF'
#!/bin/bash
echo $SECRET
EOF
chmod +x test.sh
./test.sh  # affiche une ligne vide : SECRET n'est pas exportée

export SECRET
./test.sh  # affiche 1234

readonly VERSION="1.0"
VERSION="2.0"  # erreur : VERSION en lecture seule

alias ll="ls -la"
ll
unalias ll

```

Exercice 14 — Manipulation de chaînes

```

chemin="/home/alice/projet/rapport.final.txt"
echo "${#chemin}"
echo "${chemin##*/}"          # rapport.final.txt
echo "${chemin##*.}"          # txt (plus long préfixe jusqu'au dernier point)
echo "${chemin#*.}"           # final.txt (plus court préfixe jusqu'au premier point)
echo "${chemin%.*}"           # /home/alice/projet/rapport.final
echo "${chemin/alice/bob}"
echo "${chemin^^}"■

```

Exercice 15 — Valeurs par défaut

```

echo "${ENV:-production}"
: "${ENV:=production}"
: "${API_KEY:?Erreur: API_KEY doit être défini}"■

```

Exercice 16 — Motifs de fichiers (globbing)

```

ls *.txt
ls log[12].txt
ls *. {csv,md}
ls [!l]*
mkdir -p archive_{2022..2024}

```

Exercice 17 — Tests sur fichiers et chaînes

```

[[ -f config.yml && -w config.yml ]]
[[ -d logs ]]
[[ -z "$NOM" ]]
[[ "$A" -eq "$B" ]]

```

Exercice 18 — Validation par expression régulière

```

[[ "$email" =~ ^[A-Za-z0-9.-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$ ]]
[[ "$code_postal" =~ ^[0-9]{5}$ ]]

```

Exercice 19 — Classification par condition

```
#!/bin/bash
note=$1
if (( note >= 16 )); then
    echo "Excellent"
elif (( note >= 12 )); then
    echo "Bien"
elif (( note >= 10 )); then
    echo "Passable"
else
    echo "Insuffisant"
fi
```

Exercice 20 — Boucles for

```
for i in {1..10}; do
    echo "7 x $i = $((7*i))"
done

for (( i=0; i<=20; i+=2 )); do
    echo "$i"
done

for f in *.txt; do
    echo "$f: $(wc -l < "$f") lignes"
done
```

Exercice 21 — while, until et menu select

```
i=1
while (( i <= 5 )); do
    echo "$i"
    ((i++))
done

i=1
until (( i > 5 )); do
    echo "$i"
    ((i++))
done

select choix in "Démarrer" "Arrêter" "Quitter"; do
    case $choix in
        "Quitter") echo "Au revoir"; break ;;
        *) echo "Choix: $choix" ;;
    esac
done
```

Exercice 22 — case et fonctions

```
saluer() {
    echo "Bonjour $1 !"
}
saluer "Alice"

case $extension in
    txt|pdf) echo "document" ;;
    jpg|png) echo "image" ;;
    *) echo "inconnu" ;;
esac
```

Exercice 23 — Calculs arithmétiques

```
echo $((17 % 5))
echo $((2 ** 10))
echo $(((12 + 15 + 9) / 3))
compteur=0
((compteur++)); ((compteur++)); ((compteur++))
echo $compteur
```

Exercice 24 — Tableaux indexés

```
fruits=("pomme" "banane" "cerise")
echo "${fruits[1]}"
echo "${fruits[@]}"
echo "${#fruits[@]}"
fruits+=("kiwi")
unset fruits[1]
for i in "${!fruits[@]}"; do
    echo "$i: ${fruits[$i]}"
done
```

Exercice 25 — Tableau associatif

```
declare -A capitales=( ["France"]="Paris" ["Italie"]="Rome" ["Espagne"]="Madrid" )
echo "${capitales[Italie]}"
echo "${!capitales[@]}"
echo "${#capitales[@]}"
for pays in "${!capitales[@]}"; do
    echo "$pays -> ${capitales[$pays]}"
done
unset capitales["Espagne"]
```

Exercice 26 — Signaux et nettoyage (trap)

```
#!/bin/bash
trap 'echo "Interruption détectée, nettoyage..."; exit 1' SIGINT
sleep 30
trap - SIGINT
```

Exercice 27 — Sauvegarde compressée

```
mkdir site && touch site/a.html site/b.css site/c.js
tar -czf site.tar.gz site
tar -tzf site.tar.gz
mkdir site_restore
tar -xzf site.tar.gz -C site_restore
```

Exercice 28 — Rapport d'utilisateurs (synthèse)

```
#!/bin/bash
declare -A compte
actifs=()

while IFS="," read -r nom role actif; do
    if [[ "$actif" == "oui" ]]; then
        actifs+=("$nom")
        ((compte[$role]++))
    fi
done < <(tail -n +2 users.csv)

for role in "${!compte[@]}"; do
    echo "$role: ${compte[$role]} actif(s)"
done

if [[ ${#actifs[@]} -eq 0 ]]; then
    exit 1
fi
exit 0
```

Exercice 29 — Surveillance de dossier (synthèse)

```
#!/bin/bash
dossier=$1

if [[ -z "$dossier" || ! -d "$dossier" ]]; then
    echo "Erreur: dossier invalide" >&2
    exit 1
fi

total=$(find "$dossier" -type f | wc -l)
logs=$(find "$dossier" -type f -name "*.log" | wc -l)
taille=$(du -sh "$dossier" | cut -f1)

echo "Fichiers: $total, logs: $logs, taille: $taille"

if grep -riq "ERROR" "$dossier"/*.log 2>/dev/null; then
    archive="backup_$(date +%Y-%m-%d).tar.gz"
    tar -czf "$archive" "$dossier"
    echo "Archive créée: $archive"
else
    echo "Aucune erreur détectée"
fi
```